~ Complete Step by Step ~

CSS Notes

From Zero to Hero · All Topics Covered

Index

1 Introduction & Syntax
2 Selectors
3 Pseudo-classes, Pseudo-elements, Specificity
4 Colors
5 Backgrounds
6 Text Styling
7 Fonts
8 Units & Values
9 Box Model
10 Borders, Outline, Shadow
11 Display, Visibility, Overflow, Float
12 Position & z-index
13 Lists, Links, Tables, Cursor
14 Flexbox (2 pages)
15 Grid (2 pages)
16 Transforms
17 Transitions
18 Animations & Keyframes
19 Gradients, Filters, Opacity, Images
20 Variables, Functions & Modern CSS (2 pages)
21 Responsive Design & Media Queries
22 Forms & Components
23 Best Practices & Architecture
24 Full Practice Page
25 Interview Q&A
26 Cheat Sheet

How to use these notes

→ Learn **HTML first**. CSS styles the HTML structure.
→ Read the yellow highlights – the most important points.
→ CSS code: ***property*** · **"values"** · ***selectors***. Type every example and see the result!

Notes by asadokz18 ✿

1. Introduction & Syntax

- **CSS** = **C**ascading **S**tyle **S**heets. It styles HTML: colors, fonts, spacing, layout, animation.
- **HTML** = skeleton · **CSS** = clothes & design · **JS** = behaviour.
- Made by **Håkon Wium Lie** (1996). Current standard: **CSS3** (modules) – new versions are called "CSS Level 4 modules".
- Benefits: separate design from content, reuse one file for many pages, faster, easy to maintain.

CSS Syntax

```
selector {
  property: value;
  property: value;
}
h1 { color: blue; font-size: 30px; }
```

Selector
Which HTML element to style (h1)

Declaration
property : value ;

Block
Everything inside { }

3 ways to add CSS

Way	Where	Notes
Inline	`style` attribute in the tag	One element only. Highest priority.
Internal	`<style>` tag in `<head>`	For a single page.
External	.css file linked with `<link>`	**Best** – one file for whole site.

```
<p style="color:red;">Inline</p>

<head>
  <style> p { color: blue; } </style>          <!-- Internal -->
  <link rel="stylesheet" href="style.css">     <!-- External -->
</head>
```

@import – import one CSS file into another: **`@import url("fonts.css");`** (must be at the top).

Comments

```
/* single line comment */
/* multi
   line comment */
```

Priority (Cascade) order

Browser default → External → Internal → Inline → !important

(lower priority → higher priority; if equal, the **last rule wins**)

Tip
Don't forget the **semicolon** ; after each declaration and the **closing** } – the most common CSS mistakes!

2. CSS Selectors

Selectors select the HTML elements you want to style.

Selector	Example	Selects
Universal	* { }	All elements
Type / Tag	p { }	All <p> elements
Class	.note { }	Elements with class="note"
ID	#logo { }	The element with id="logo" (unique)
Grouping	h1, h2, p { }	All listed elements (comma)
Multiple classes	.a.b { }	Has **both** class a and b
Tag + class	p.big { }	<p> that has class big

```
* { margin: 0; padding: 0; box-sizing: border-box; }
h1, h2 { color: navy; }
.highlight { background: yellow; }
#header { background: black; color: white; }
```

Combinators (relationship selectors)

Combinator	Example	Meaning
Descendant (space)	div p { }	All <p> **inside** div (any level)
Child (>)	div > p { }	Only **direct children** p of div
Adjacent (+)	h1 + p { }	The **first p right after** h1
General sibling (~)	h1 ~ p { }	**All p siblings** after h1

```
<div>
  <p>Direct child</p>
  <section>
    <p>Nested p</p>
  </section>
</div>
```

Try it

div p → both p

div > p → only "Direct child"

Class vs ID

.class

Reusable - many elements. An element can have many classes: class="a b c"

#id

Unique - only one element per page. Higher priority than class.

Selector list tricks

```
.card .title { }        /* title inside card        */
ul li:first-child { }   /* first li of each ul      */
nav a, footer a { }     /* links in nav AND footer  */
```

Best practice

Use **classes** for styling. Avoid styling by id and avoid very long selector chains.

3. Attribute, Pseudo & Specificity

Attribute selectors

Selector	Selects
`[target]`	Elements that have the attribute
`[type="text"]`	Attribute equals exactly
`[class~="a"]`	Attribute contains the whole word "a"
`[lang|="en"]`	Equals "en" or starts with "en-"
`[href^="https"]`	Starts with
`[href$=".pdf"]`	Ends with
`[href*="google"]`	Contains anywhere

Pseudo-classes (state of an element) – : single colon

- :hover – mouse over
- :active – being clicked
- :focus – input selected
- :visited / :link – links
- :checked :disabled :enabled
- :required :optional :valid :invalid
- :first-child :last-child
- :nth-child(n) – 2, odd, even, 3n
- :nth-of-type(n) • :only-child
- :not(.x) – everything except
- :empty :root • :is() :where() :has()
- :focus-within

```
a:hover { color: red; }
li:nth-child(odd) { background: #eee; }
input:focus { border: 2px solid blue; }
p:not(.note) { color: gray; }
```

Link order (LoVe HAte): :link → :visited → :hover → :active

Pseudo-elements – :: double colon

- ::before – add content before
- ::after – add content after
- ::first-letter – drop cap
- ::first-line
- ::selection – selected text
- ::placeholder • ::marker

```
p::first-letter { font-size: 40px; color: red; }
h1::before { content: "★ "; color: orange; }
::selection { background: yellow; }
```

Specificity – who wins?

Inline (1000) > #id (100) > .class / [attr] / :pseudo-class (10) > tag / ::pseudo-element (1) > * (0)

```
p { color: blue; }          /* 1   */
.note { color: green; }     /* 10  */
p.note { color: red; }      /* 11  */
#intro { color: purple; }   /* 100 → wins */
```

- Same specificity → the **last** rule wins. !important beats everything (avoid it!).
- Inheritance: children take some properties from parents (color, font, text-align…). Not inherited: margin, padding, border, background. Force with ***inherit***, ***initial***, ***unset***.

4. Colors

CSS colors are used with **color**, **background-color**, **border-color**, etc.

Ways to write a color

Format	Example	Result
Name	red, blue, tomato, gold, navy… (140 names)	tomato gold
HEX	#RRGGBB → **#ff0000** · short **#f00** · with alpha **#ff000080**	#7b2cbf
RGB	**rgb(255, 99, 71)** – each 0 to 255	rgb
RGBA	**rgba(0, 0, 0, 0.5)** – alpha 0 (clear) to 1	50% black
HSL	**hsl(120, 100%, 40%)** – hue 0–360°, saturation, lightness	hsl
HSLA	**hsla(240, 100%, 50%, 0.3)**	hsla
Keywords	**transparent** · **currentColor** (takes text color)	

```
h1 { color: #1f4fd8; }
p  { color: rgb(40, 40, 40); background-color: rgba(255, 255, 0, 0.3); }
a  { color: hsl(200, 80%, 45%); }
```

Color wheel facts

RGB (screens)

Red + Green + Blue light.
rgb(0,0,0) = **black**
rgb(255,255,255) = **white**

HSL hue

0° red · 60° yellow · 120° green · 180° cyan · 240° blue · 300° magenta

Color-related properties

- → **color** – text color.
- → **background-color** – box color.
- → **opacity** – transparency of the **whole element** (0 to 1).
- → **accent-color** – color of checkbox, radio, range.
- → **caret-color** – blinking cursor in inputs. **border-color**, **outline-color**, **text-shadow** color…

rgba vs opacity

rgba() makes only the color transparent. **opacity** makes the **whole element including text** transparent.

Tip

Good contrast between text & background is needed for readability (WCAG ratio 4.5:1). Pick 3–5 colors and use CSS variables.

5. Backgrounds

Property	Values / meaning
background-color	Any color: **lightblue**, #fff, rgba()…
background-image	**url("pic.jpg")** or a gradient
background-repeat	**repeat** (default) · repeat-x · repeat-y · **no-repeat** · space · round
background-position	left / center / right + top / center / bottom · or **50% 20px**
background-size	auto · **cover** (fill, may crop) · **contain** (fit all) · 100px 50px · 50%
background-attachment	**scroll** (default) · **fixed** (parallax) · local
background-origin	Where image starts: padding-box · border-box · content-box
background-clip	Where background is painted: border-box · padding-box · content-box · **text**
background-blend-mode	multiply, screen, overlay… mix image & color

```
body {
  background-color: #fff8e1;
  background-image: url("pattern.png");
  background-repeat: no-repeat;
  background-position: center top;
  background-size: cover;
  background-attachment: fixed;
}
```

Shorthand background

Order: **color image repeat attachment position / size**

```
div { background: #eee url("bg.jpg") no-repeat fixed center / cover; }
```

Multiple backgrounds

Separate with commas – the **first one is on top.**

```
.hero {
  background: url("stars.png") top left no-repeat,
              url("sky.jpg") center / cover;
}
```

Cool effects

Hero image overlay

```
.hero {
  background:
   linear-gradient(rgba(0,0,0,.5), rgba(0,0,0,.5)),
   url("hero.jpg") center / cover;
}
```

Gradient text

```
h1 {
  background: linear-gradient(red, blue);
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
}
```

Remember

Always set a **background-color** as backup in case the image fails to load. Use **cover** for full-screen hero images.

6. Text Styling

Property	Values & meaning
color	Text color.
text-align	left · right · center · **justify** · start · end
text-decoration	none · underline · overline · line-through · with color / style: ***underline wavy red***
text-transform	uppercase · lowercase · **capitalize** · none
text-indent	Indent of first line: ***30px***
letter-spacing	Gap between letters: ***3px***
word-spacing	Gap between words: ***10px***
line-height	Height of a line: ***1.5*** (unitless is best)
text-shadow	***x y blur color*** → 2px 2px 4px gray
white-space	normal · nowrap · pre · pre-wrap · pre-line
word-break / overflow-wrap	break-all · keep-all · ***break-word*** – break long words
text-overflow	**ellipsis** "..." (needs overflow:hidden + nowrap)
direction / unicode-bidi	ltr · rtl – for Urdu / Arabic text
vertical-align	baseline · top · middle · bottom · sub · super
hyphens · tab-size · writing-mode	auto hyphenation · tab width · vertical text

```
h1 {
  color: navy;
  text-align: center;
  text-transform: uppercase;
  letter-spacing: 2px;
  text-shadow: 2px 2px 5px rgba(0,0,0,.4);
}
a { text-decoration: none; }
p { line-height: 1.6; text-indent: 30px; text-align: justify; }
```

Single-line "..." ellipsis trick

```
.title { width: 200px; white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
```

Preview

SHADOW TEXT

Wavy underline

Tip

Use **line-height: 1.5–1.7** for body text – much more readable.

7. Fonts

Font families

- Five generic families: **serif** (Times – with tails) · **sans-serif** (Arial – clean) · **monospace** (Courier – code) · **cursive** · **fantasy**.
- Write a **font stack**: first choice, fallback…, generic last. Quote names with spaces.

```
body { font-family: "Segoe UI", Arial, Helvetica, sans-serif; }
code { font-family: "Courier New", monospace; }
```

Property	Values
font-size	px, em, rem, %, vw · keywords: small, medium, large, x-large · default 16px
font-weight	normal (400) · bold (700) · 100 – 900 · lighter · bolder
font-style	normal · **italic** · oblique
font-variant	normal · small-caps
font-stretch	condensed … expanded (if font supports)
font	**Shorthand** – style variant weight size/line-height family

```
p { font: italic small-caps bold 18px/1.5 Georgia, serif; }
```

Shorthand rule

In **font** shorthand only **font-size and font-family are required**, and they must be last.

Google Fonts (web fonts)

```
<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;700&display=swap" rel="stylesheet">
```

```
body { font-family: "Poppins", sans-serif; }
```

@font-face – your own font file

```
@font-face {
  font-family: "MyFont";
  src: url("fonts/myfont.woff2") format("woff2"),
       url("fonts/myfont.woff") format("woff");
  font-weight: 400;
  font-display: swap;
}
h1 { font-family: "MyFont", sans-serif; }
```

Font formats

WOFF2 (best, small) · WOFF · TTF · OTF

Tips

Use max 2 fonts per site. **font-display: swap** shows text immediately.

Serif: Hello

Sans-serif: Hello

Monospace: Hello

Cursive: Hello

8. Units & Values

Absolute units (fixed size)

Unit	Meaning
px	Pixel – most used on screens
pt / pc	Point (1/72 inch) / pica (12pt) – print
cm mm in	Centimeter, millimeter, inch – print

Relative units (flexible)

Unit	Relative to...
%	Percentage of the **parent** element
em	**Parent's font-size** (2em = twice parent). Compounds when nested!
rem	**Root (html) font-size** – default 16px → 1.5rem = 24px. Predictable ✔
vw / vh	1% of viewport **width** / **height** (50vw = half screen)
vmin / vmax	1% of smaller / larger viewport side
ch / ex	Width of "0" / height of "x" of the font
fr	Fraction of free space – Grid only
dvh svh lvh	Dynamic / small / large viewport height (mobile browsers)

```
html { font-size: 16px; }
h1   { font-size: 2.5rem; }     /* 40px */
.hero{ height: 100vh; width: 80%; }
p    { margin-bottom: 1.5em; }
```

Other value types

- → **Numbers:** 0, 1.5, opacity: .5
- → **Strings:** content: "Hi"
- → **URL:** url("img.png")
- → **Angle:** deg, rad, turn → 45deg
- → **Time:** s, ms → 0.3s, 300ms
- → **Colors:** red, #fff, rgb()
- → **Keywords:** auto, inherit, none...
- → **Functions:** calc() var() min() max() clamp()

Which to use?

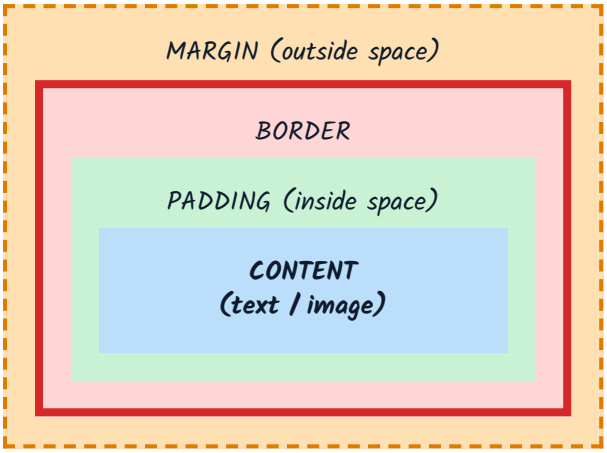
rem for fonts & spacing • **%** / **vw** for widths • **px** for borders & small details • **em** for things that scale with their own text (buttons).

Remember

0 needs no unit: ***margin: 0***. Don't put a space between number and unit: ***10px ✔ 10 px ✘***.

9. The Box Model

Every HTML element is a **box** made of 4 layers (inside → outside):



- **Content** – text, image.
- **Padding** – space around content, **inside** the border (background shows).
- **Border** – line around padding.
- **Margin** – space **outside** border (always transparent).

Width & height

- ***width / height***, ***min-width / max-width***, ***min-height / max-height*** – set content size (auto by default).
- Total width = width + padding (L+R) + border (L+R) + margin (L+R) (content-box).

Padding & Margin shorthand

```
.box { margin: 10px;              } /* all 4 sides              */
.box { margin: 10px 20px;         } /* top-bottom | left-right  */
.box { margin: 10px 20px 30px;    } /* top | left-right | bottom */
.box { margin: 10px 20px 30px 40px; }/* top right bottom left ⟳ clockwise */
.box { margin: 0 auto; }            /* center a block horizontally ✔ */
.box { padding-left: 15px; margin-top: -5px; } /* single sides, negative margin ok */
```

Margin collapse

Vertical margins of adjacent blocks **merge** into the bigger one (20px + 30px = 30px). Horizontal margins don't. Flex/grid items don't collapse.

auto margin

margin: 0 auto centers a block only if it has a **width**.

box-sizing – very important!

Value	Meaning
content-box	(default) width = content only; padding & border are **added** → box gets bigger.
border-box	width **includes** padding + border → easy to calculate ✔

```
*, *::before, *::after { box-sizing: border-box; }   /* use this in every project */
```

- ***overflow***, ***outline***, ***box-shadow*** do **not** change the box size. Inline elements ignore vertical margin/width/height.

10. Borders, Outline & Shadows

Border

Property	Values
border-width	thin · medium · thick · 3px
border-style	solid · dashed · dotted · double · groove · ridge · inset · outset · none · hidden
border-color	Any color
border-top / -right / -bottom / -left	One side only (also -width -style -color)
border (shorthand)	**width style color** → 2px solid red

solid dashed dotted double groove ridge

border-radius – rounded corners

```
.card   { border: 2px solid #333; border-radius: 12px; }
.circle { width: 100px; height: 100px; border-radius: 50%; }   /* circle */
.leaf   { border-radius: 30px 0 30px 0; }   /* TL TR BR BL */
```

Outline

- Line drawn **outside** the border; does **not take space**. Properties: outline-width, outline-style, outline-color, **outline-offset**.
- Used for focus rings: ***input:focus { outline: 2px solid blue; }*** · never remove focus outline without replacement!

box-shadow

Syntax: ***offset-x offset-y blur spread color [inset]***

```
.card  { box-shadow: 0 4px 12px rgba(0,0,0,.25); }
.glow  { box-shadow: 0 0 15px 5px skyblue; }
.inner { box-shadow: inset 0 0 10px #000; }
.multi { box-shadow: 0 2px 4px red, 0 8px 16px blue; }   /* many shadows */
```

soft shadow glow inset outline

Remember

Text has **text-shadow**; boxes have **box-shadow**; images with transparent PNG use **filter: drop-shadow()**.

11. Display, Visibility, Overflow & Float

display

Value	Meaning
block	New line, full width; width/height work (div, p).
inline	Same line; width/height/vertical margin ignored (span, a).
inline-block	Same line **and** width/height work.
none	Element removed – takes **no space**.
flex / inline-flex	Flexbox container (chapter 14).
grid / inline-grid	Grid container (chapter 15).
table, table-cell...	Behave like table parts · **contents** · list-item

visibility & opacity

- ***visibility: hidden*** – invisible but **still takes space**. ***display: none*** – gone, no space.
- ***opacity: 0*** – invisible, takes space and still clickable.

overflow – content bigger than its box

Value	Meaning
visible	(default) content spills outside
hidden	extra content is clipped
scroll	always show scrollbars
auto	scrollbar only when needed ✔

Also ***overflow-x***, ***overflow-y***. Similar: ***resize: both***, ***scroll-behavior: smooth***.

```
.box { width: 200px; height: 100px; overflow: auto; }
```

float & clear (old layout)

- ***float***: left · right · none – pushes element to the side; text wraps around it (images in articles).
- ***clear***: left · right · both – element must move below floated ones.
- Floated children make the parent **collapse** (height 0) → fix with the **clearfix**:

```
img { float: left; margin: 0 10px 10px 0; }
.footer { clear: both; }
.clearfix::after { content: ""; display: table; clear: both; }
```

Note

Don't use float for page layouts today – use **Flexbox** or **Grid**. Use float only to wrap text around images.

Centering – cheat list

- Block: ***margin: 0 auto*** (with width)
- Text: ***text-align: center***
- Flex: ***justify-content + align-items: center***
- Grid: ***place-items: center***

12. Position & z-index

The **position** property + **top / right / bottom / left** offsets control where an element sits.

Value	Behaviour
static	(default) normal flow. top/left/z-index have **no effect**.
relative	Stays in flow; offsets move it **from its own normal position**. Space is still reserved. Used as a **reference parent** for absolute children.
absolute	Removed from flow; positioned relative to the **nearest positioned ancestor** (not static), else the page.
fixed	Removed from flow; stays at the **same place in the viewport** even when scrolling (sticky headers, chat buttons).
sticky	Acts like relative until you scroll to a threshold (top: 0), then sticks like fixed.

```
.parent { position: relative; }
.badge  { position: absolute; top: 10px; right: 10px; }   /* corner of .parent */

.navbar { position: fixed; top: 0; left: 0; width: 100%; }
.th     { position: sticky; top: 0; background: white; }
.center { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }
.cover  { position: absolute; inset: 0; }   /* top/right/bottom/left = 0 */
```



z-index – stacking order

- Higher **z-index** = nearer to you (on top). Default **auto** (0). Negative values go behind.
- Works **only on positioned elements** (relative / absolute / fixed / sticky) and flex/grid items.
- Elements with same z-index: the later one in HTML is on top.
- A new **stacking context** is created by position+z-index, opacity<1, transform, filter – children can't escape it.

```
.modal   { position: fixed; z-index: 1000; }
.overlay { position: fixed; inset: 0; z-index: 999; background: rgba(0,0,0,.5); }
```

Remember

absolute → look for a **position:relative parent** · use small z-index numbers (1, 10, 100) · sticky needs a **top** value.

13. Lists, Links, Tables & Cursor

Styling lists

Property	Values
list-style-type	disc · circle · square · decimal · lower-alpha · upper-roman · none
list-style-position	outside (default) · inside
list-style-image	url("star.png") – custom bullet
list-style	Shorthand: **square inside**

```
ul { list-style-type: square; }
nav ul { list-style: none; margin: 0; padding: 0; display: flex; gap: 20px; }  /* horizontal menu */
```

Styling links – LoVe HAte order

```
a:link    { color: blue; }
a:visited { color: purple; }
a:hover   { color: red; text-decoration: underline; }
a:active  { color: orange; }
.btn { display: inline-block; padding: 8px 18px; background: #1f4fd8; color: #fff; border-radius: 6px; text-decoration: none; }
```

Styling tables

Property	Use
border-collapse	**collapse** (single border) · separate
border-spacing	Gap between cells (separate mode)
caption-side	top · bottom
empty-cells	show · hide
table-layout	auto · fixed (faster, equal columns)
padding, text-align, vertical-align	Inside cells

```
table { border-collapse: collapse; width: 100%; }
th, td { border: 1px solid #333; padding: 8px; }
tr:nth-child(even) { background: #f2f2f2; }
tr:hover { background: #dbe6ff; }
```

cursor – mouse pointer shape

default · **pointer** 👆 (links, buttons) · **text** · **move** · **wait** · **help** · **not-allowed** · **crosshair** · **grab / grabbing** · **zoom-in** · **none**

```
button { cursor: pointer; }   button:disabled { cursor: not-allowed; }
```

Tip

Other useful: **user-select: none** (no text select) · **pointer-events: none** (ignore clicks).

14. Flexbox – Part 1 (Container)

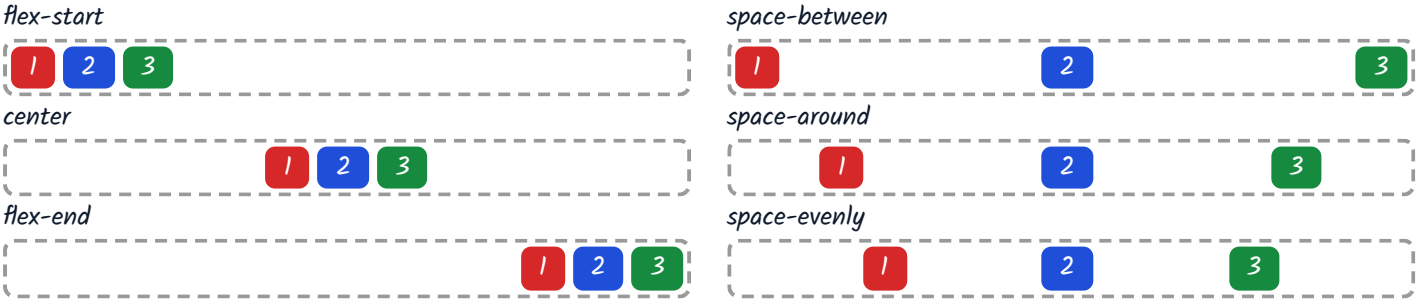
→ **Flexbox** = one-dimensional layout (a **row OR a column**). Great for navbars, cards, centering.
→ Parent = **flex container** · children = **flex items**. Start with ***display: flex***.




```
.container { display: flex; }
```

Container property	Values
flex-direction	**row** · row-reverse · column · column-reverse
flex-wrap	**nowrap** · wrap · wrap-reverse
flex-flow	Shorthand: ***row wrap***
justify-content	Align on **main axis**: flex-start · flex-end · center · space-between · space-around · space-evenly
align-items	Align on **cross axis** (one line): stretch · flex-start · flex-end · center · baseline
align-content	Space between **multiple lines** (needs wrap)
gap / row-gap / column-gap	Space between items: ***gap: 20px***

justify-content visual



Perfect centering

```
.center { display: flex; justify-content: center; align-items: center; height: 100vh; }
.navbar { display: flex; justify-content: space-between; align-items: center; }
```

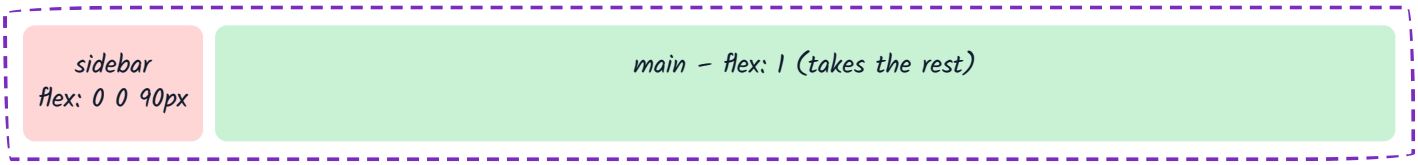
Remember

direction = **column** swaps the axes: justify-content now works vertically, align-items horizontally.

14. Flexbox – Part 2 (Items)

Item property	Meaning
flex-grow	How much the item **grows** to fill free space (default 0). **flex-grow: 1** = share equally.
flex-shrink	How much it **shrinks** when space is small (default 1). 0 = never shrink.
flex-basis	Starting size before grow/shrink: auto · 200px · 30%
flex	**Shorthand** grow shrink basis → **flex: 1** (= 1 1 0) · **flex: 0 0 200px** · **flex: auto** · none
align-self	Override align-items for **one item**: auto · flex-start · center · flex-end · stretch
order	Change visual order (default 0). Lower = first. Negative ok.

```
.sidebar { flex: 0 0 250px; }      /* fixed 250px, doesn't grow or shrink */
.main    { flex: 1; }              /* takes all remaining space           */
.item:last-child { margin-left: auto; }  /* push last item to far right  */
.special { align-self: flex-end; order: -1; }
```



Wrapping cards (responsive grid-like)

```
.cards { display: flex; flex-wrap: wrap; gap: 16px; }
.card  { flex: 1 1 250px; }        /* grow, shrink, min ~250px → wraps automatically */
```

Common flex patterns

Navbar

Logo left, links right: ***justify-content: space-between***

Sticky footer

body: ***display:flex; flex-direction:column; min-height:100vh;*** main: ***flex:1***

Equal-height cards

Default ***align-items: stretch*** makes all items the same height.

Gotcha

float, clear, vertical-align have no effect on flex items. Use ***min-width: 0*** to let long text shrink.

Flex vs Grid

Flexbox = **1D** (row or column), content-driven. Grid = **2D** (rows AND columns), layout-driven. They work great together!

15. CSS Grid – Part 1 (Container)

→ **Grid** = **two-dimensional** layout: rows and columns together. Start with ***display: grid***.

```css
.grid {
  display: grid;
  grid-template-columns: 200px 1fr 1fr;   /* 3 columns */
  grid-template-rows: 80px auto;          /* 2 rows    */
  gap: 16px;
}
```

200px	1fr	1fr
item 4	item 5	item 6

Container property	Meaning
grid-template-columns / rows	Size of tracks: px, %, **fr**, auto, minmax(), repeat()
gap (row-gap, column-gap)	Space between cells
grid-template-areas	Name regions of the layout (see below)
grid-auto-rows / -columns	Size of automatically created tracks
grid-auto-flow	row (default) · column · dense
justify-items / align-items	Align **items inside cells** (horizontal / vertical): start · end · center · stretch
place-items	Shorthand of align + justify → ***center***
justify-content / align-content	Align the **whole grid** in the container

Helpful functions

```css
grid-template-columns: repeat(3, 1fr);                  /* 3 equal columns                */
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); /* responsive, no media query ✔ */
grid-template-columns: 250px minmax(0, 1fr);            /* fixed + flexible               */
```

The fr unit

1fr = one **fraction** of the free space. ***1fr 2fr*** → second column is twice as wide as the first.

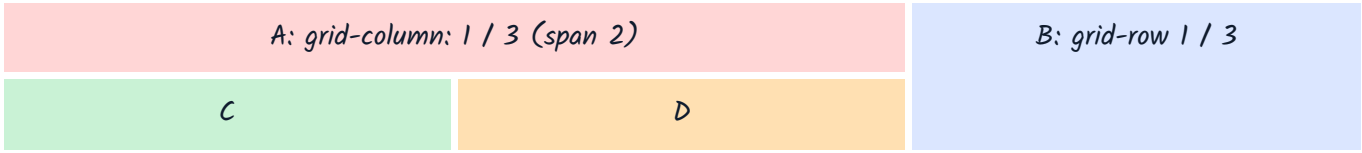
Named template areas – page layout

```css
.page {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
}
.header  { grid-area: header; }   .sidebar { grid-area: sidebar; }
.main    { grid-area: main; }     .footer  { grid-area: footer; }
```

15. CSS Grid – Part 2 (Items)

Item property	Meaning
grid-column	Start / end column lines: **1 / 3** or **span 2**
grid-row	Start / end row lines: **2 / 4** or **span 2**
grid-area	Name of an area, or row-start / col-start / row-end / col-end
justify-self	Align this item horizontally in its cell
align-self	Align this item vertically in its cell
place-self	Shorthand of both
order / z-index	Visual order · overlap items

Grid lines are numbered from 1




```
.a { grid-column: 1 / 3; }          /* from line 1 to line 3          */
.b { grid-column: 3; grid-row: 1 / 3; }
.wide { grid-column: span 2; }
.full { grid-column: 1 / -1; }      /* -1 = last line → full width   */
```

Responsive card gallery (very popular)

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 20px;
}
```

auto-fit vs auto-fill

auto-fill keeps empty columns. **auto-fit** collapses empty columns so items stretch.

minmax(min, max)

Track is never smaller than min and never bigger than max.

Implicit vs explicit grid

- **Explicit** = tracks you defined with template-columns/rows. **Implicit** = extra rows created automatically; size them with ***grid-auto-rows: 100px***.
- **Subgrid** (***grid-template-columns: subgrid***) lets a child grid use the parent's tracks.

When to use what?

Grid – whole page layout, galleries, dashboards. **Flexbox** – navbars, buttons rows, centering, small components.

16. Transforms (2D & 3D)

The **transform** property moves, rotates, scales or skews an element **without affecting other elements' layout**.

2D functions

Function	Effect
translate(x, y)	Move. translateX(50px), translateY(-20px). Percent = of own size.
rotate(angle)	Rotate: **rotate(45deg)**, negative = anticlockwise
scale(x, y)	**scale(1.5)** bigger · scale(0.5) smaller · scaleX · scaleY · scale(-1) flip
skew(x, y)	Slant: **skewX(20deg)**
matrix(a,b,c,d,e,f)	All 2D transforms in one function

```
.box { transform: rotate(10deg) scale(1.2) translateX(20px); }   /* many, applied right → left */
.card:hover { transform: translateY(-8px) scale(1.03); }
```



transform-origin

The point around which the transform happens (default **center**): **transform-origin: top left;** or **0 0**, **50% 100%**.

3D transforms

Property / function	Meaning
rotateX() rotateY() rotate2()	Rotate around an axis (flip a card: rotateY(180deg))
translate2() · scale3d()	Move nearer / farther
perspective	Depth distance on parent: **perspective: 800px** (smaller = stronger 3D)
transform-style	flat · **preserve-3d** (children keep 3D)
backface-visibility	hidden – hide the back of a flipped element

```
.scene { perspective: 800px; }
.card  { transform-style: preserve-3d; transition: transform .6s; }
.scene:hover .card { transform: rotateY(180deg); }   /* flip card */
```

Tip

Transforms are GPU-accelerated → smooth animations. Individual properties also exist: **translate**, **rotate**, **scale**.

17. Transitions

Transitions make a property change **smoothly over time** (instead of instantly) – usually on **:hover**, **:focus** or when a class is added by JavaScript.

Property	Meaning
transition-property	Which property: color, background, transform, **all** (default)
transition-duration	How long: **0.3s** or 300ms (required!)
transition-timing-function	Speed curve: **ease** (default) · linear · ease-in · ease-out · ease-in-out · cubic-bezier(…) · steps(4)
transition-delay	Wait before starting: 0.2s
transition	**Shorthand**: property duration timing delay

```
.btn {
  background: #1f4fd8;
  color: white;
  transition: background 0.3s ease, transform 0.2s ease-out;
}
.btn:hover { background: #d62828; transform: scale(1.08); }
```

Timing functions – speed curves

- → **linear** – constant speed
- → **ease** – slow → fast → slow
- → **ease-in** – slow start
- → **ease-out** – slow end
- → **ease-in-out** – slow both
- → **steps(n)** – jumps in n steps

Common transition effects

```
a       { transition: color .3s; }
img     { transition: transform .4s; }   img:hover { transform: scale(1.1); }
.menu   { max-height: 0; overflow: hidden; transition: max-height .4s; }
.menu.open { max-height: 300px; }
.fade   { opacity: 0; transition: opacity .5s; }   .fade.show { opacity: 1; }
```

Important rules

- → Put **transition on the normal state** (not only hover) so it animates both in and out.
- → **display** cannot be transitioned; use opacity / visibility.

Performance

Animate **transform** and **opacity** only – they are fastest. Avoid animating width/height/top/left.

Also

New: **transition-behavior: allow-discrete** and **@starting-style** allow animating display and entry animations in modern browsers.

18. Animations & @keyframes

Animations run **automatically** and can have many steps (unlike transitions that need a trigger and have only start → end).

Step 1: define with @keyframes

```
@keyframes slide {
  from { transform: translateX(0);     background: red;  }
  to   { transform: translateX(200px); background: blue; }
}
@keyframes bounce {          /* using percentages = many steps */
  0%   { transform: translateY(0);     }
  50%  { transform: translateY(-40px); }
  100% { transform: translateY(0);     }
}
```

Step 2: apply with animation properties

Property	Values
animation-name	Name of the @keyframes
animation-duration	2s (required)
animation-timing-function	ease, linear, ease-in-out, steps()...
animation-delay	Wait before start (negative = start midway)
animation-iteration-count	1 (default) · 3 · **infinite**
animation-direction	normal · reverse · **alternate** · alternate-reverse
animation-fill-mode	none · forwards (keep last frame) · backwards · both
animation-play-state	running · paused
animation	**Shorthand** – name duration timing delay count direction fill-mode

```
.ball { animation: bounce 1s ease-in-out infinite; }
.box  { animation: slide 2s linear 0.5s 3 alternate forwards; }
.ball:hover { animation-play-state: paused; }
```

Popular animations

```
@keyframes spin  { to { transform: rotate(360deg); } }        /* loader  */
.loader { width:40px; height:40px; border:5px solid #ddd; border-top-color:#1f4fd8;
          border-radius:50%; animation: spin 1s linear infinite; }

@keyframes pulse { 50% { transform: scale(1.15); opacity: .7; } }
@keyframes fadeIn{ from { opacity: 0; transform: translateY(20px); } }
```

Transition vs Animation

Transition: 2 states, needs a trigger. Animation: many steps, runs by itself, can loop.

Accessibility

Respect users who dislike motion: **@media (prefers-reduced-motion: reduce) { * { animation: none; } }**

19. Gradients, Filters, Opacity & Images

Gradients (used in background-image)

Type	Example
linear-gradient	**linear-gradient(to right, red, yellow)** · angle: **45deg** · color stops: **red 0%, blue 70%**
radial-gradient	**radial-gradient(circle, white, black)** – from the center outward
conic-gradient	**conic-gradient(red, yellow, green, red)** – around a point (pie charts)
repeating-*	**repeating-linear-gradient(45deg, #ddd 0 10px, #fff 10px 20px)** – stripes




```
.hero { background: linear-gradient(135deg, #1f4fd8, #7b2cbf); }
```

opacity & filter

Property	Meaning
opacity	0 (invisible) … 1 (solid) – whole element incl. children
filter	blur(5px) · brightness(1.2) · contrast(150%) · grayscale(100%) · sepia() · invert() · hue-rotate(90deg) · saturate() · **drop-shadow**(2px 2px 4px #000)
backdrop-filter	Filters what is **behind** the element → glass effect: **blur(10px)**
mix-blend-mode	multiply, screen, overlay, difference…

```
img { filter: grayscale(100%) blur(1px); }   img:hover { filter: none; }
.glass { background: rgba(255,255,255,.25); backdrop-filter: blur(10px); }
```

Images & media

Property	Meaning
max-width: 100%	Responsive image (with **height: auto**)
object-fit	fill · **cover** (crop) · **contain** · none · scale-down – how image fits its box
object-position	Which part of image shows: center top
aspect-ratio	**16 / 9** · 1 / 1 – keep proportions
clip-path	Cut shapes: **circle(50%)** · polygon(50% 0, 100% 100%, 0 100%)
mask-image	Hide parts using an image / gradient

```
.avatar { width: 120px; aspect-ratio: 1; object-fit: cover; border-radius: 50%; }
.tri    { clip-path: polygon(50% 0, 100% 100%, 0 100%); }
```

20. Variables & CSS Functions

CSS Custom Properties (variables)

→ Names start with `--` · declared in a selector (global = `:root`) · used with `var(--name)` · they **inherit** and can be changed with JavaScript.

```
:root {
  --main-color: #1f4fd8;
  --radius: 10px;
  --space: 16px;
}
.btn  { background: var(--main-color); border-radius: var(--radius); padding: var(--space); }
.card { --main-color: #d62828; }                  /* override inside .card only       */
.text { color: var(--missing, black); }           /* fallback value if not defined    */
```

```
document.documentElement.style.setProperty("--main-color", "green");   "C">// change with JS
```

Dark mode with variables

```
:root { --bg: #fff; --text: #111; }
@media (prefers-color-scheme: dark) { :root { --bg: #111; --text: #eee; } }
body { background: var(--bg); color: var(--text); }
```

CSS functions

Function	Use
`calc()`	Math with mixed units: **`width: calc(100% - 250px)`** (spaces around + and – needed!)
`min(a, b)`	Smaller value: **`width: min(90%, 800px)`**
`max(a, b)`	Bigger value: **`font-size: max(16px, 2vw)`**
`clamp(min, ideal, max)`	**`font-size: clamp(1rem, 2.5vw, 2rem)`** - fluid, no media query
`var()`	Use a custom property
`attr()`	Read an HTML attribute: **`content: attr(title)`**
`url()` · `rgb()` · `hsl()`	Resources and colors
`repeat()` · `minmax()` · `fit-content()`	Grid sizing
`counter()`	Automatic numbering with counter-reset & counter-increment

```
h1 { font-size: clamp(1.8rem, 5vw, 3.5rem); }
.container { width: min(100% - 2rem, 1100px); margin-inline: auto; }
body { counter-reset: chapter; }  h2::before { counter-increment: chapter; content: "Chapter " counter(chapter) ": "; }
```

20b. Modern CSS Features

Logical properties

Work with writing direction (good for RTL like Urdu): **margin-inline**, **margin-block**, **padding-inline-start**, **inset-inline**, **border-inline-end**, **inline-size** (= width), **block-size** (= height).

CSS Nesting

```
.card {
  padding: 16px;
  .title { font-weight: bold; }
  &:hover { box-shadow: 0 4px 12px rgba(0,0,0,.3); }
}
```

Cascade layers & @supports

```
@layer reset, base, components;          /* later layers win, no matter the specificity */
@layer base { body { font-family: sans-serif; } }

@supports (display: grid) { .wrap { display: grid; } }          /* feature query */
@supports not (backdrop-filter: blur(4px)) { .glass { background: #fff; } }
```

Container queries

Style by the **size of the parent container**, not the screen.

```
.wrapper { container-type: inline-size; }
@container (min-width: 400px) { .card { display: flex; } }
```

Multi-column layout

```
.article { column-count: 3; column-gap: 30px; column-rule: 2px solid #ccc; }   /* or column-width: 200px */
h2 { column-span: all; }
```

Scroll features

```
html { scroll-behavior: smooth; }
.slider { display: flex; overflow-x: auto; scroll-snap-type: x mandatory; }
.slide  { flex: 0 0 100%; scroll-snap-align: start; }
.top    { scroll-margin-top: 80px; }          /* offset for sticky header */
```

:has() – parent selector

.card:has(img) – card that contains an image.
form:has(:invalid)

Other new things

accent-color · **color-mix()** · **text-wrap: balance** · **@property** · **view-transitions** · **scroll-timeline**

Check support

Use **caniuse.com** before using very new features, and give a fallback.

21. Responsive Design & Media Queries

Responsive design = one website that looks good on **phone, tablet and desktop**.

The 4 steps

1. Add the **viewport meta tag** in HTML:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

2. Use **flexible units** (%, rem, vw, fr) and ***max-width*** instead of fixed widths.
3. Make **images flexible**: ***img { max-width: 100%; height: auto; }***
4. Use **media queries** to change layout at breakpoints.

Media query syntax

```
@media screen and (max-width: 768px) {
  .menu  { display: none; }
  .grid  { grid-template-columns: 1fr; }
}
@media (min-width: 1024px) { .container { width: 1000px; } }
@media (min-width: 600px) and (max-width: 900px) { ... }
@media (orientation: landscape) { ... }
@media print { nav, footer { display: none; } }
@media (hover: hover) { .btn:hover { background: red; } }
@media (prefers-color-scheme: dark) { ... }
@media (prefers-reduced-motion: reduce) { ... }
```

Common breakpoints

Device	Width	Device	Width
Small phone	≤ 480px	Laptop	992 – 1199px
Phone	481 – 767px	Desktop	≥ 1200px
Tablet	768 – 991px	Large screens	≥ 1440px

Mobile-first approach ✔

→ Write the **mobile styles first**, then add ***min-width*** queries for bigger screens (less code, faster on phones).

```
.cards { display: grid; gap: 16px; }                    /* mobile: 1 column */
@media (min-width: 600px)  { .cards { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 1000px) { .cards { grid-template-columns: repeat(4, 1fr); } }
```

Responsive images

HTML: ***srcset***, **<picture>**. CSS: max-width, object-fit, background-size: cover.

No media query needed

flex-wrap · grid auto-fit/minmax · clamp() · min() · container queries

22. Forms & Common Components

Styling forms

```
input, select, textarea {
  width: 100%; padding: 10px; margin: 6px 0 14px;
  border: 2px solid #ccc; border-radius: 8px; font: inherit; box-sizing: border-box;
}
input:focus   { border-color: #1f4fd8; outline: none; box-shadow: 0 0 0 3px rgba(31,79,216,.25); }
input:invalid { border-color: #d62828; }
input[type="checkbox"], input[type="radio"] { accent-color: #1f4fd8; width: auto; }
input::placeholder { color: #999; }
button[type="submit"] { background: #1f4fd8; color: #fff; padding: 10px 24px; border: none; border-radius:
8px; cursor: pointer; }
```

Horizontal navbar

```
.nav { display: flex; justify-content: space-between; align-items: center; background: #0F1B2D; padding: 0 20px;
position: sticky; top: 0; }
.nav ul { display: flex; gap: 20px; list-style: none; }
.nav a  { color: #fff; text-decoration: none; padding: 14px 8px; display: block; }
.nav a:hover { color: #E08F00; }
```

Card

```
.card { border-radius: 12px; overflow: hidden; box-shadow: 0 4px 14px rgba(0,0,0,.15); transition: transform .3s; }
.card:hover { transform: translateY(-6px); }
.card img { width: 100%; height: 180px; object-fit: cover; }
.card .body { padding: 16px; }
```

Dropdown menu

```
.dropdown { position: relative; display: inline-block; }
.dropdown-content { display: none; position: absolute; top: 100%; left: 0; min-width: 160px; background: #fff;
box-shadow: 0 8px 16px rgba(0,0,0,.2); z-index: 10; }
.dropdown:hover .dropdown-content { display: block; }
```

Tooltip & Modal centering

```
.tip { position: relative; }
.tip::after { content: attr(data-tip); position: absolute; bottom: 120%; left: 50%; transform: translateX(-50%);
              background: #333; color: #fff; padding: 4px 8px; border-radius: 4px; opacity: 0; transition: opacity
.2s; }
.tip:hover::after { opacity: 1; }
.modal { position: fixed; inset: 0; display: grid; place-items: center; background: rgba(0,0,0,.6); }
```

Hamburger menu

Hide menu on mobile with a media query + toggle a class with JavaScript.

Other

Buttons · badges · alerts · tabs · accordions (details) · progress bars

23. Best Practices & Architecture

Good habits

- Use **external** stylesheets.
- Use **classes**, keep specificity low.
- Avoid !important & ID selectors.
- Use **border-box** + a small reset.
- Use variables for colors & spacing.
- **Mobile-first** media queries.
- Use rem / % instead of fixed px sizes.
- Group related rules; comment sections.
- Minify CSS for production.
- Test in many browsers & devices.

CSS reset / normalize

```
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
html { font-size: 100%; scroll-behavior: smooth; }
body { line-height: 1.6; font-family: system-ui, sans-serif; }
img, video { max-width: 100%; display: block; }
a { color: inherit; }
```

Naming: BEM (Block__Element--Modifier)

```
.card { }                /* Block    */
.card__title { }         /* Element  */
.card--featured { }      /* Modifier */
.btn--large { }
```

Suggested file / code order

Variables → Reset → Base (body, headings) → Layout (header, grid) → Components (btn, card) → Utilities → Media queries

Property order inside a rule

Layout (display, position) → Box (width, margin, padding, border) → Typography → Visual (color, background) → Other (transition, cursor).

Tools & ecosystem

Tool	What
DevTools (F12)	Inspect & edit CSS live in the browser
Sass / SCSS	Preprocessor: variables, nesting, mixins, partials → compiles to CSS
Bootstrap / Tailwind	CSS frameworks: ready components / utility classes
Autoprefixer	Adds vendor prefixes: -webkit- -moz- -ms-
Validator	jigsaw.w3.org/css-validator

```
$primary: #1f4fd8;                /* SCSS example */
.btn { background: $primary; &:hover { background: darken($primary, 10%); } }
```

Performance

Fewer, smaller CSS files · avoid heavy selectors & huge shadows/filters · animate transform/opacity only · use **will-change** sparingly.

24. Full Practice Page

A small responsive landing page using flexbox, grid, variables, hover & media queries.

index.html

```
<header class="nav"><h2>asadokz18</h2><nav><a href="#about">About</a> <a href="#skills">Skills</a></nav>
</header>
<section class="hero"><h1>Hello, I'm learning CSS</h1><a href="#skills" class="btn">See my skills</a></section>
<section id="skills" class="cards">
  <div class="card"><h3>HTML</h3><p>Structure</p></div>
  <div class="card"><h3>CSS</h3><p>Style</p></div>
  <div class="card"><h3>JS</h3><p>Behaviour</p></div>
</section>
```

style.css

```
:root { --blue: #1f4fd8; --dark: #0F1B2D; --gold: #E08F00; }
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: system-ui, sans-serif; line-height: 1.6; color: var(--dark); }

.nav { display: flex; justify-content: space-between; align-items: center;
       padding: 12px 24px; background: var(--dark); color: #fff; position: sticky; top: 0; }
.nav a { color: #fff; text-decoration: none; margin-left: 16px; transition: color .3s; }
.nav a:hover { color: var(--gold); }

.hero { min-height: 60vh; display: grid; place-content: center; gap: 20px; text-align: center; color: #fff;
        background: linear-gradient(135deg, var(--blue), #7b2cbf); }
.btn  { background: var(--gold); color: #fff; padding: 12px 28px; border-radius: 30px;
        text-decoration: none; justify-self: center; transition: transform .2s; }
.btn:hover { transform: scale(1.08); }

.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 20px; padding: 40px
24px; }
.card  { padding: 24px; border-radius: 14px; box-shadow: 0 4px 14px rgba(0,0,0,.15); transition: transform .3s; }
.card:hover { transform: translateY(-8px); }

@media (max-width: 600px) { .nav { flex-direction: column; gap: 8px; } h1 { font-size: 1.6rem; } }
```

Practice tasks

① Style your HTML resume ② Make a responsive navbar ③ Build a pricing-card grid ④ Make a loading spinner ⑤ Clone a simple landing page ⑥ Make a dark-mode toggle with variables.

25. Interview Questions & Answers

Q1. What is CSS?

Cascading Style Sheets – a language to style and lay out HTML pages (colors, fonts, spacing, responsiveness).

Q2. Three ways to add CSS?

Inline (style attribute), internal (<style> tag), external (.css file via <link>). External is best.

Q3. Explain the box model.

Content + padding + border + margin. **box-sizing: border-box** includes padding & border in the width.

Q4. What is specificity?

The weight deciding which rule wins: inline > id > class/attr/pseudo-class > tag. Equal → last rule wins. !important overrides.

Q5. Class vs ID?

Class is reusable; id is unique and has higher specificity.

Q6. display: none vs visibility: hidden vs opacity: 0?

none – removed, no space. hidden – invisible but keeps space. opacity 0 – invisible, keeps space, still clickable.

Q7. Explain position values.

static (default), relative (offset from itself), absolute (to nearest positioned ancestor), fixed (to viewport), sticky (relative then fixed on scroll).

Q8. Flexbox vs Grid?

Flexbox is 1-dimensional (row or column); Grid is 2-dimensional (rows and columns).

Q9. em vs rem?

em is relative to the parent's font-size; rem relative to the root (html) font-size.

Q10. Pseudo-class vs pseudo-element?

Pseudo-class (:hover) = state of an element. Pseudo-element (::before) = a part of an element or generated content.

Q11. Transition vs animation?

Transition: smooth change between two states with a trigger. Animation: multi-step with @keyframes; can run automatically and loop.

Q12. How to center a div?

Flex: justify-content & align-items center • Grid: place-items center • Block: margin 0 auto • Absolute: top/left 50% + translate(-50%,-50%).

Q13. What are media queries? What is mobile-first?

@media rules that apply styles by screen conditions. Mobile-first = base styles for phones, then min-width queries for larger screens.

Q14. What is margin collapsing? What are CSS variables?

Adjacent vertical margins merge into the larger one. Variables: --name declared in :root, used with var(--name).

Q15. What is z-index? What is the cascade? What is a CSS preprocessor?

z-index = stacking order of positioned elements. Cascade = rules resolved by origin, specificity and order. Preprocessor (Sass) adds variables, nesting, mixins and compiles to CSS.

26. Quick Cheat Sheet

Selectors

`*` all · `p` tag · `.c` class · `#i` id · `a, b` group · `a b` descendant · `a > b` child · `a + b` next · `a ~ b` siblings · `[attr]` · `:hover` `::before`

Text & Font

color · font-family · font-size · font-weight · font-style · line-height · text-align · text-decoration · text-transform · letter-spacing · text-shadow · white-space

Box Model

width · height · min/max · margin · padding · border · border-radius · box-sizing · outline · box-shadow · overflow

Background

background-color · -image · -repeat · -position · -size · -attachment · gradients · opacity · filter

Layout

display · position + top/left/right/bottom · z-index · float · clear · flex · grid · gap · margin: 0 auto · columns

Flexbox

container: flex-direction, flex-wrap, justify-content, align-items, gap
item: flex, flex-grow, flex-shrink, flex-basis, align-self, order

Grid

container: grid-template-columns/rows/areas, gap, place-items
item: grid-column, grid-row, grid-area, place-self

Motion

transform · transition · @keyframes · animation · transform-origin · perspective · will-change

Units & functions

`px` `%` `em` `rem` `vw` `vh` `fr` `deg` `s` `ms` · `calc()` `var()` `min()` `max()` `clamp()` `rgb()` `hsl()` `url()` `repeat()` `minmax()` `linear-gradient()`

Learning roadmap

HTML → CSS → Flexbox & Grid → Responsive → JavaScript → Sass / Tailwind → Projects!

Final advice

Inspect websites with DevTools (F12), copy ideas, rebuild them from scratch, and make one small project every week. 💪

~ Thank you! Keep learning ~

by asadokz18 ✿